GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY
Department of Information Technology

**Mini Project**

# SwiftML

## Accelerating machine learning journeys.

Under the esteemed guidance of

***- P.Bharathi***

***Assistant Professor***

*D.Harshitha 22241A1280*
*K.Sruthika 22241A1296*
*K.Varsha 22241A1295*

**Batch No: B3**

# Abstract

SwiftML is an interesting tool designed to simplify the process of automated machine learning models. It is a tool that is accessible even to non-experts. It is a user-friendly tool where users can evaluate the performance of different algorithms based on their datasets and choose a best-suited model. SwiftML performs essential tasks such as preprocessing, model training and testing. SwiftML supports both supervised and unsupervised learning tasks. With an intuitive Streamlit interface, users can easily upload datasets, choose algorithms, and view real-time results. The Pickle library integration allows users to save and download trained models for offline use, making it ideal for production deployment or personal use.SwiftML supports a wide range of 10-15 machine-learning algorithms. After training, it provides detailed performance metrics such as Mean Absolute Error, R2, Mean Square Error, and Root Mean Square Error, which helps users easily compare and select the best model. Using the Pycaret library, SwiftML processes the training and evaluating models, which helps save time and effort.SwiftML has become a valuable resource for data scientists, analysts, and anyone looking to implement machine learning models without any domain experts or extensive coding.

**Keywords**: Automated machine-learning, Pycaret, Streamlit, Pickle.

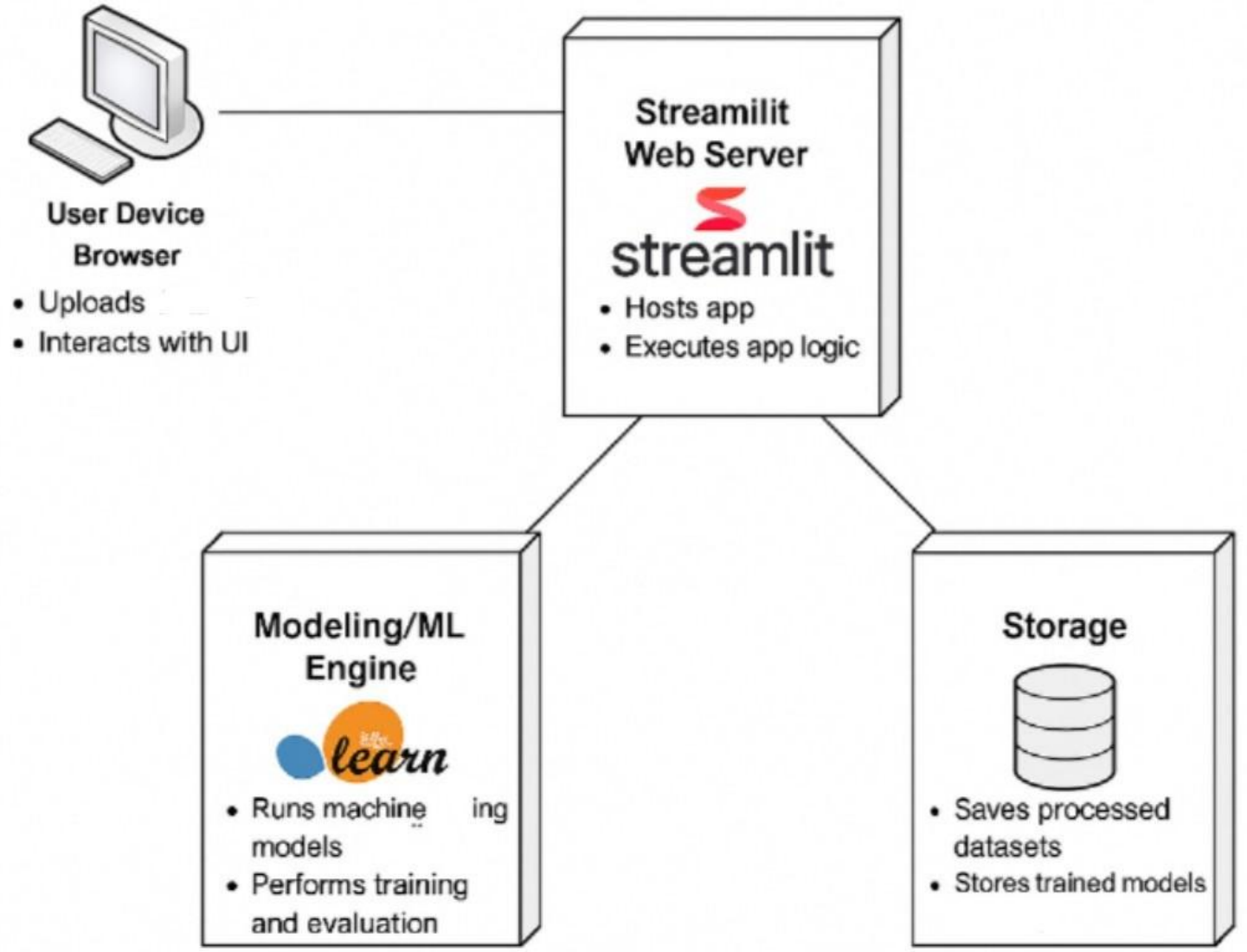
# Literature Survey

| Ref No | Author and Journal/Year | Methodology | Advantages | Disadvantages |
|---|---|---|---|---|
| [1] | Shubhra Kanti Karmaker Santu Md. Mahadi Hassan ArXiv, 2020 | Proposed a seven-tier taxonomy to classify AutoML systems based on their autonomy. | Identifies manual bottlenecks in AutoML, examines challenges in achieving full automation. | Taxonomy may not generalize well; lacks practical implementation details. |
| [2] | Xiang Wang *Volume 212, 5 January 2021 | AutoML techniques, categorizing them by pipeline stages. | Enables ML application in diverse domains with minimal expert knowledge. | Limited real-world applicability; lacks domain-specific refinements. |
| [3] | Journal of Information and Intelligence 2024 | Analyzed data processing requirements and existing NAS algorithms. | Improves model selection and hyperparameter tuning. | No empirical validation; outdated AutoML strategies. |

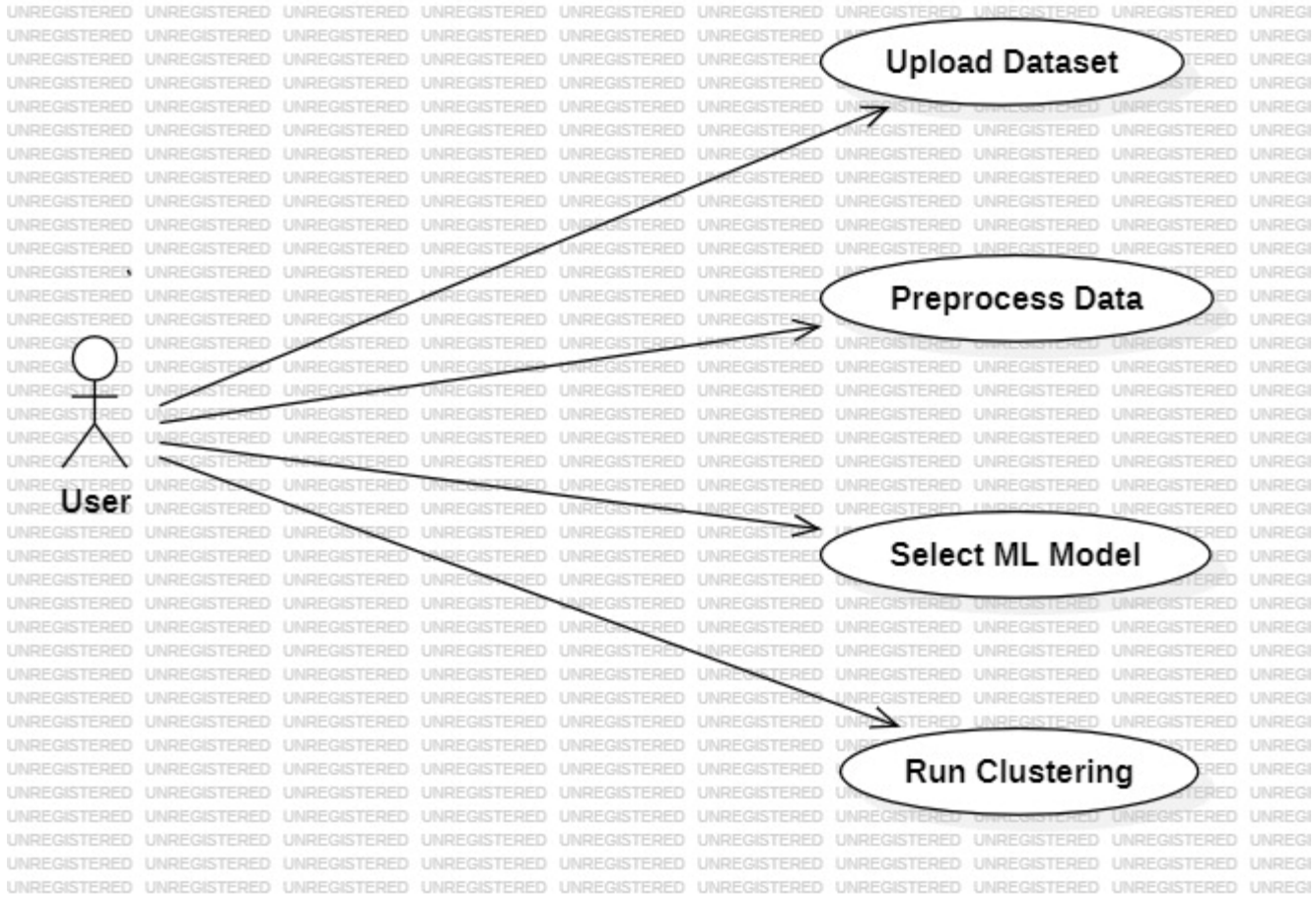
# Literature Survey

| Ref No | Author and Journal/Year | Methodology | Advantages | Disadvantages |
|---|---|---|---|---|
| [4] | Youngho Park<br>Journal of Minimally Invasive Surgery,2024 | Offering code-based instructions, to demonstrate the application of AutoML tools in R. | Automates complex tasks such as data preprocessing, model selection, and hyperparameter tuning. | R-specific; not user-friendly for non-programmers. |
| [5] | Gilbert Lim<br>Daniel Ting<br>Scientific Reports,2024 | AutoKeras (AK) is compared against custom deep learning models. | AutoKeras performs well compared to handcrafted deep learning models. | AutoKeras may not always outperform manual models;<br>risk of overfitting. |
| [6] | Springer,2024 | AutoML automates the process of model creation by defining a search space, using a search strategy, optimizing | AutoML makes machine learning accessible, efficient, and optimized, reducing manual effort and enabling | Limited flexibility in complex, domain-specific problems and may require manual fine-tuning for optimal results. |

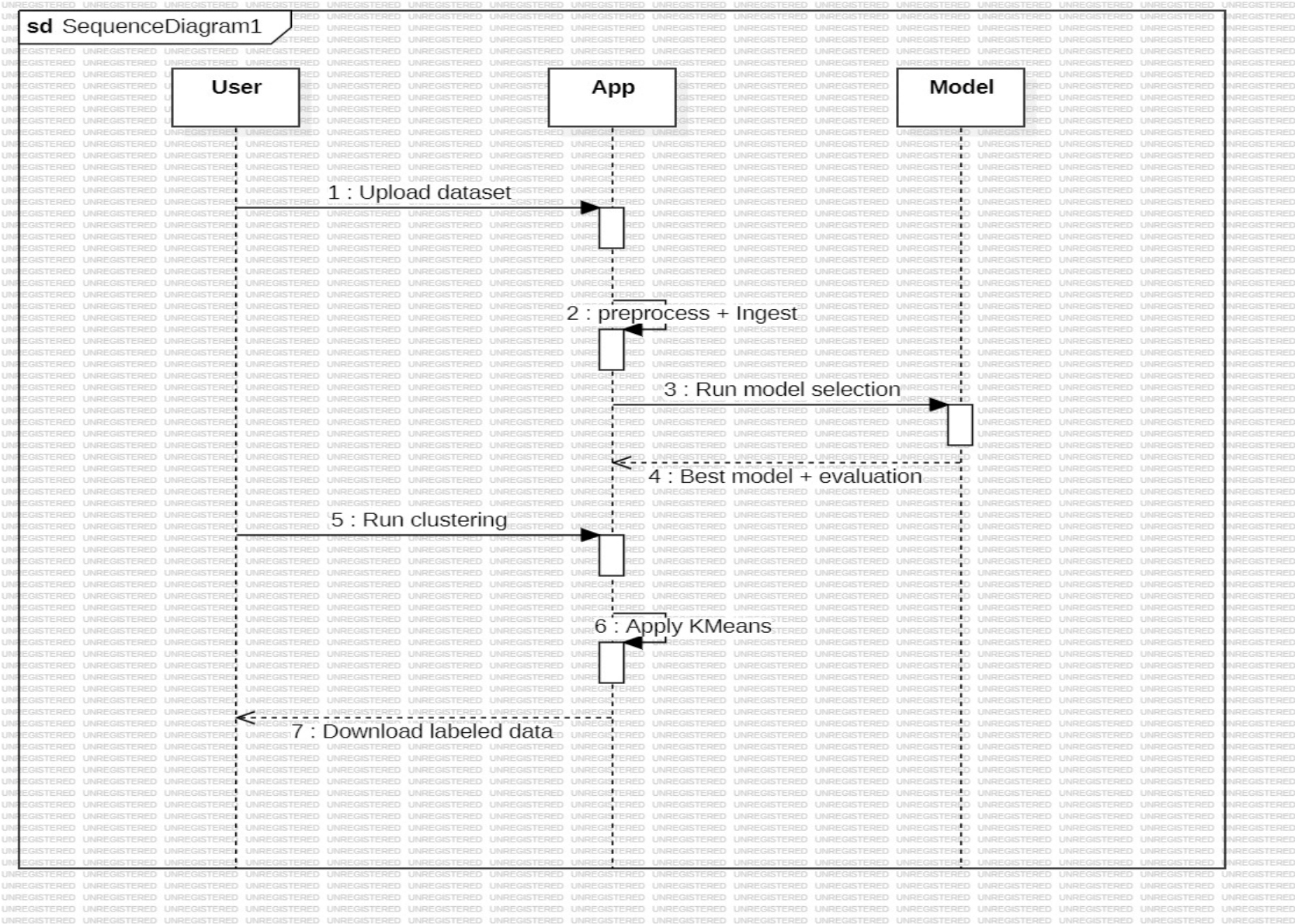
# System Architecture



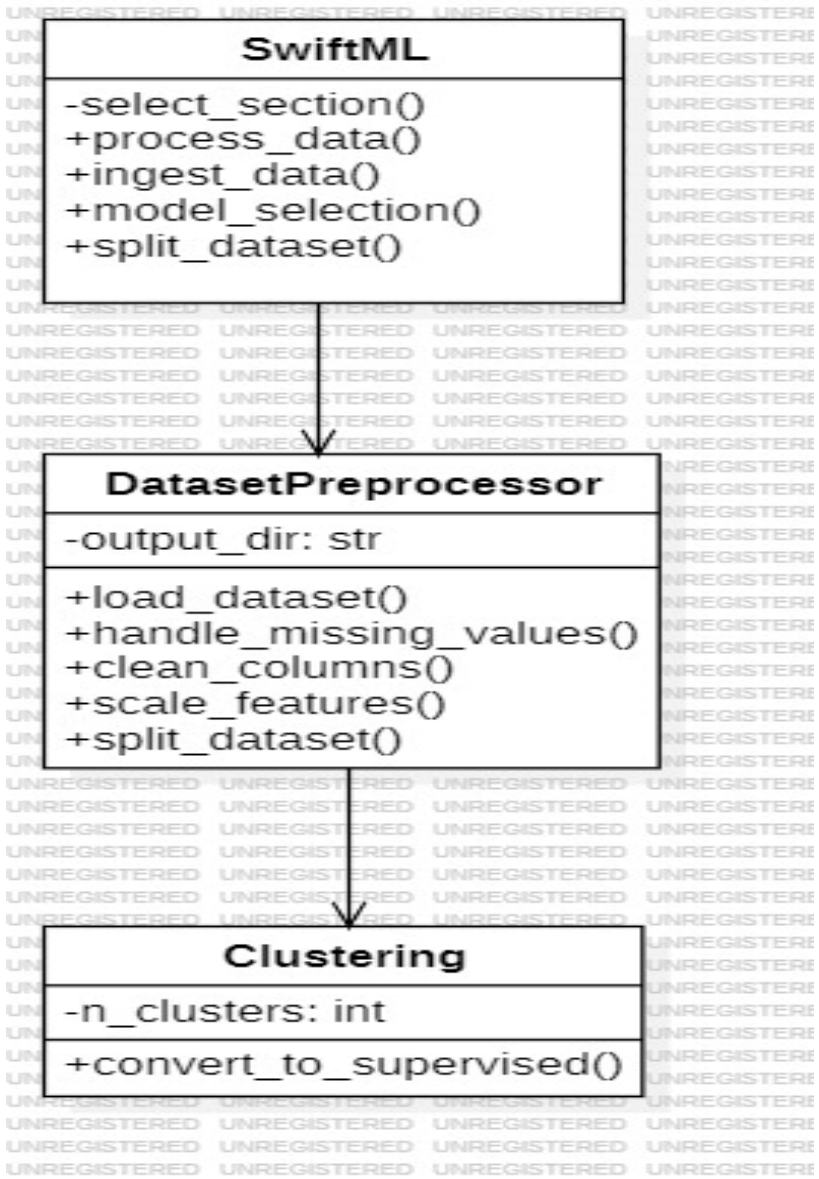
# Use case Diagram



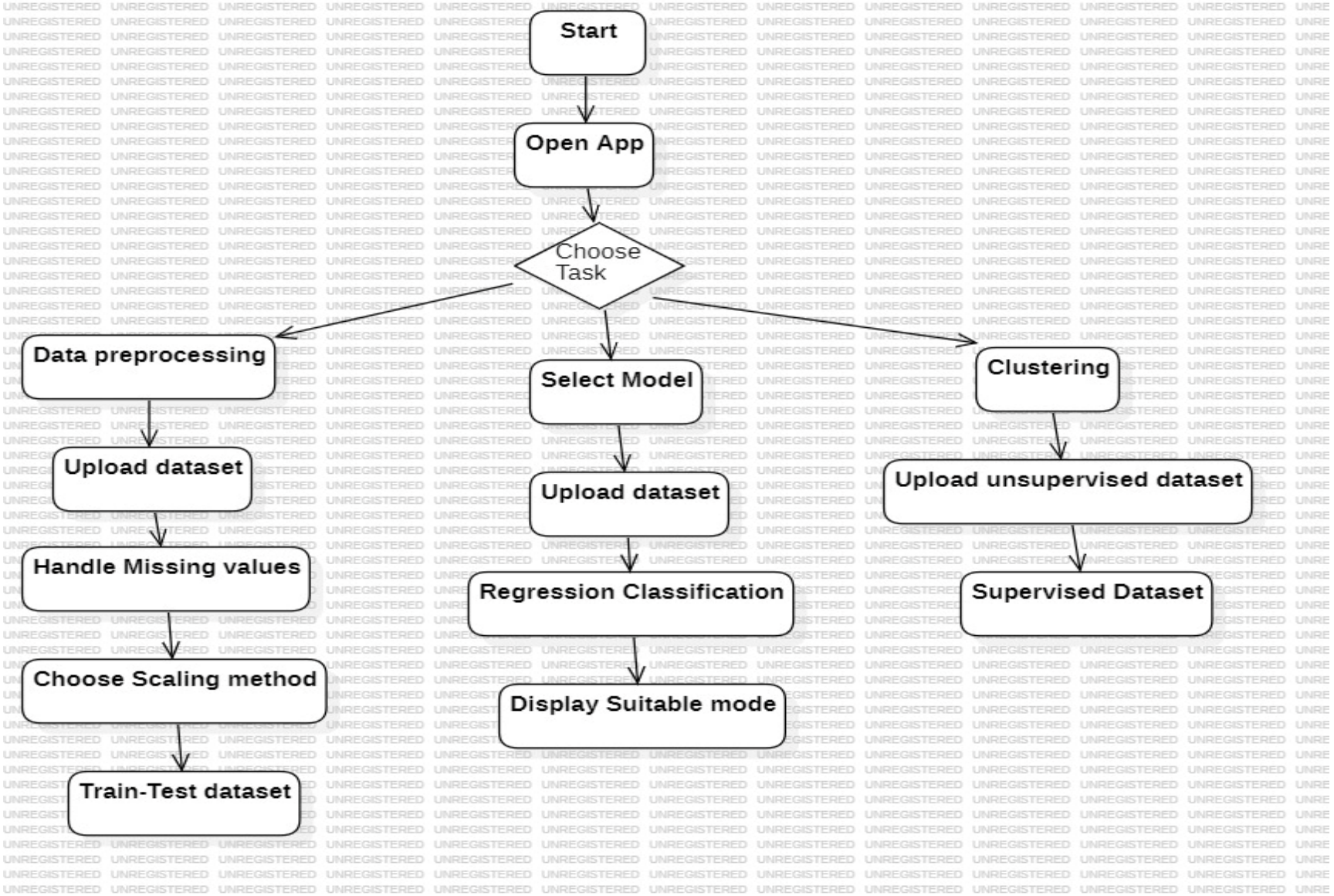
# Sequence Diagram



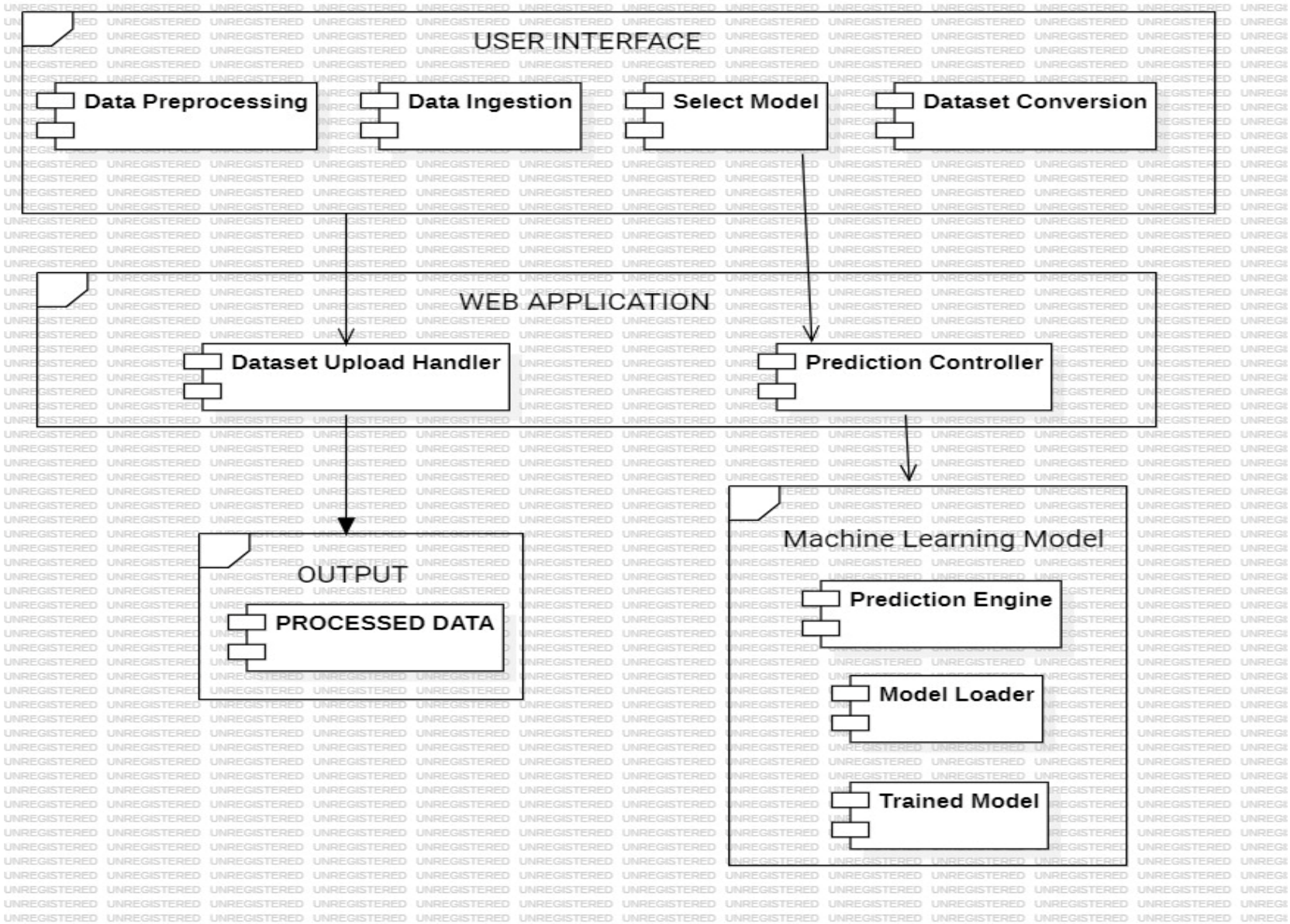
# Class Diagram



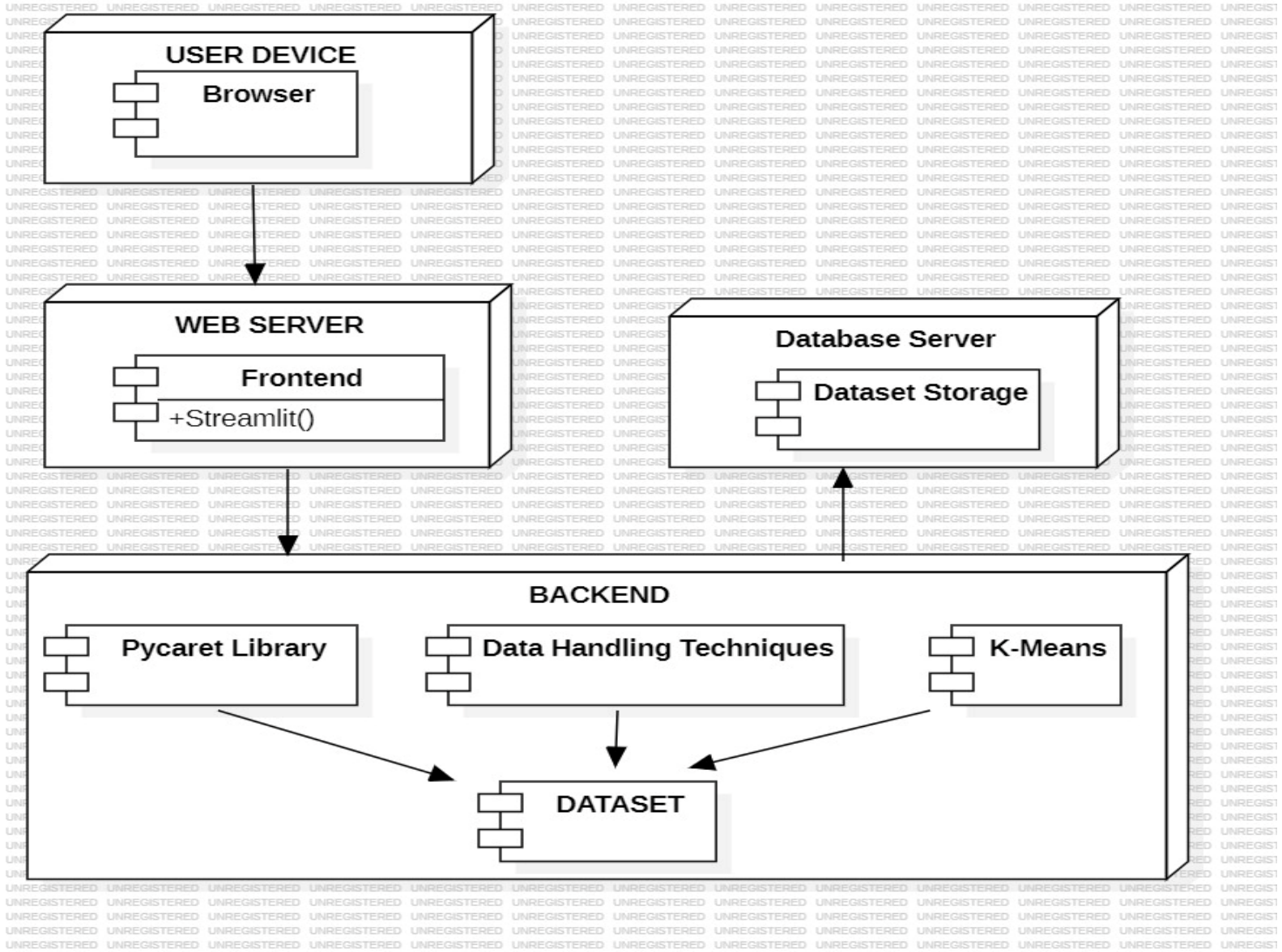
# Activity Diagram



# Component Diagram



# Deployment Diagram



# Existing System

The existing machine learning system often requires users' advanced knowledge of algorithms, data preparation and model assessment. This can be an obstacle to non-experts who want to take advantage of machine learning for practical applications. Typically, the process consists of several stages such as functional technique, model choices and Hyperpieme setting, which can be complex and time-consuming. Existing automatic tools can offer simplified interfaces, but they still need some technical understanding. SwiftML wants to cope with these challenges by offering a simple, user-friendly platform that streamlines the machine learning workflow, making it available to a wider audience.

1. Google AutoML

• Professionals: High accuracy, easy integration with cloud services, minimal coding.

• Opposition: Payment service, Internet access requires, limited adaptation.

2. H2O AutoML

• Professionals: Open sources, many ML models, support good performance.

• Opposition: Complex user interface, heavy setup, not favourable for beginners.

3. Auto-vicar

• Professionals: Professional model choices and attitude, open-SUS.

• Opposition: No graphic interface, python skills, or complex setup is required.

4. Package

• Professionals: Low codes, many models, support time savings.

• Opposition: No GUI, users must write code, not interactive for beginners.

# Proposed System

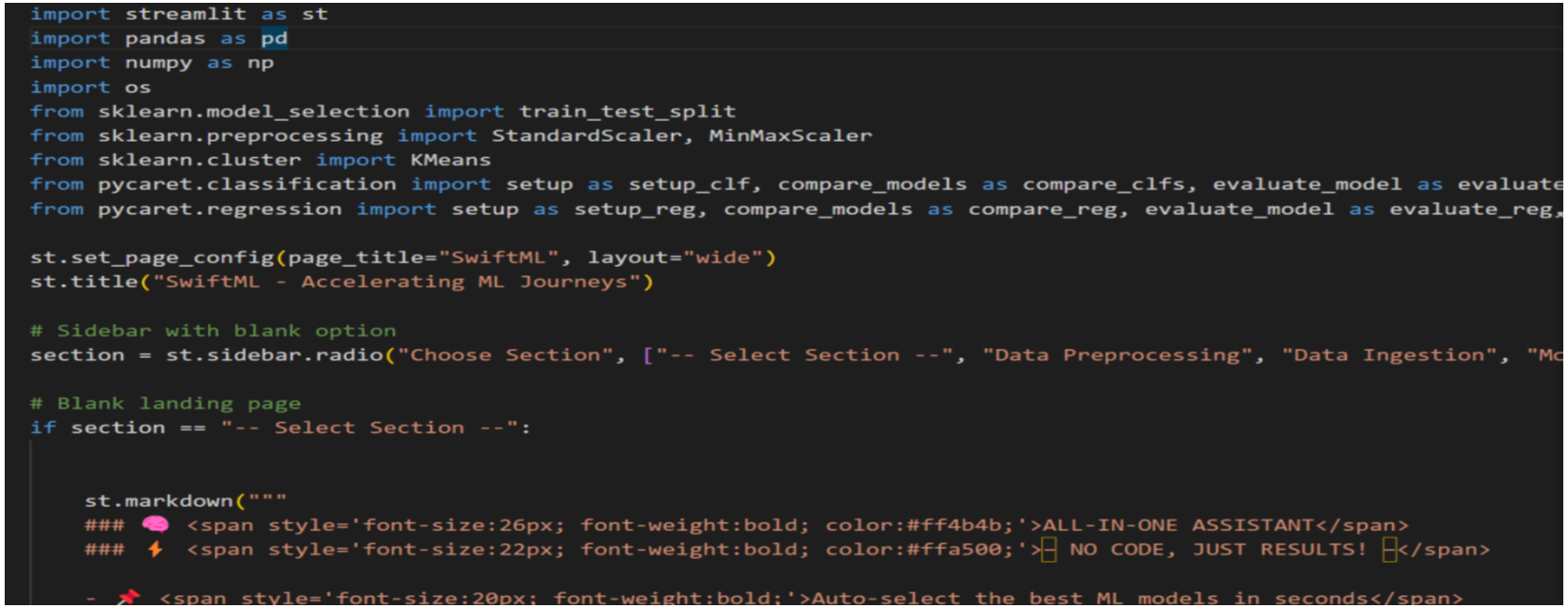
The proposed gadget, SwiftML – Accelerating Machine Learning Journeys, is designed to simplify the device getting-to-know technique for non-specialists. It utilizes Streamlit as the frontend interface and integrates PyCaret, an open-source machine gaining knowledge of library, together with Pickle for model serialization. SwiftML automates key stages of system mastering, including information preprocessing, version selection, training, assessment, and deployment. Users can absolutely upload their datasets, choose their target variable, and allow SwiftML to deal with the relaxation, from data cleaning to providing excellent version recommendations. The system provides a clean-to-apprehend interface, getting rid of the need for technical expertise at the same 4

time as retaining flexibility and high performance, and making machine studying greater accessible and quicker for people and organizations.

# Implementation

**Library Imports:**

- Imports Streamlit for building the web interface.
- Uses pandas and NumPy for data handling and manipulation.
- Imports preprocessing modules (MinMaxScaler, StandardScaler) and clustering algorithms (KMeans, DBSCAN) from sklearn.
- Uses pycaret.classification for automated model training and comparison.



```
import streamlit as st
import pandas as pd
import numpy as np
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.cluster import KMeans
from pycaret.classification import setup as setup_clf, compare_models as compare_clfs, evaluate_model as evaluate
from pycaret.regression import setup as setup_reg, compare_models as compare_reg, evaluate_model as evaluate_reg,

st.set_page_config(page_title="SwiftML", layout="wide")
st.title("SwiftML - Accelerating ML Journeys")

# Sidebar with blank option
section = st.sidebar.radio("Choose Section", ["-- Select Section --", "Data Preprocessing", "Data Ingestion", "Mo

# Blank landing page
if section == "-- Select Section --":


    st.markdown("""
    ### 🧠 <span style='font-size:26px; font-weight:bold; color:#ff4b4b;'>ALL-IN-ONE ASSISTANT</span>
    ### ⚡ <span style='font-size:22px; font-weight:bold; color:#ffa500;'>🚫 NO CODE, JUST RESULTS! 🚫</span>

    - 📌 <span style='font-size:20px; font-weight:bold;'>Auto-select the best ML models in seconds</span>
```

# Preprocessing:

- Users upload a CSV file.
- Choose between MinMaxScaler or StandardScaler.
- The app processes the data accordingly and displays the scaled output.

```python
class DatasetPreprocessor:
    def load_dataset(self, path: str) -> pd.DataFrame:
        if path.endswith('.csv'):
            return pd.read_csv(path)
        elif path.endswith('.json'):
            return pd.read_json(path)
        elif path.endswith(('.xlsx', '.xls')):
            return pd.read_excel(path)
        else:
            raise ValueError(f"Unsupported format: {path}")

    def handle_missing_values(self, df: pd.DataFrame, strategy: str = 'mean') -> pd.DataFrame:
        numeric_cols = df.select_dtypes(include=[np.number]).columns
        strategies = {
            'mean': lambda x: x.fillna(x.mean(numeric_only=True)),
            'median': lambda x: x.fillna(x.median(numeric_only=True)),
            'mode': lambda x: x.fillna(x.mode().iloc[0]),
            'interpolate': lambda x: x.interpolate()
        }
        df[numeric_cols] = strategies[strategy](df[numeric_cols])
        string_cols = df.select_dtypes(include=['object', 'string']).columns
        for col in string_cols:
            if df[col].isnull().any():
                mode_val = df[col].mode().iloc[0] if not df[col].mode().empty else 'unknown'
                df[col] = df[col].fillna(mode_val)
```

(parameter) strategy: str

## Data Ingestion:

- Allows CSV upload.
- Displays the uploaded dataset using a data table.
- Reports the shape of the dataset(number of rows and columns).

```python
# === Data Ingestion Section ===
elif section == "Data Ingestion":
    file = st.file_uploader("📤 Upload your file")
    if file:
        df = pd.read_csv(file)
        st.dataframe(df)
        df.to_csv("sourcefile.csv", index=False)
        st.success("Data saved to `sourcefile.csv`")
```

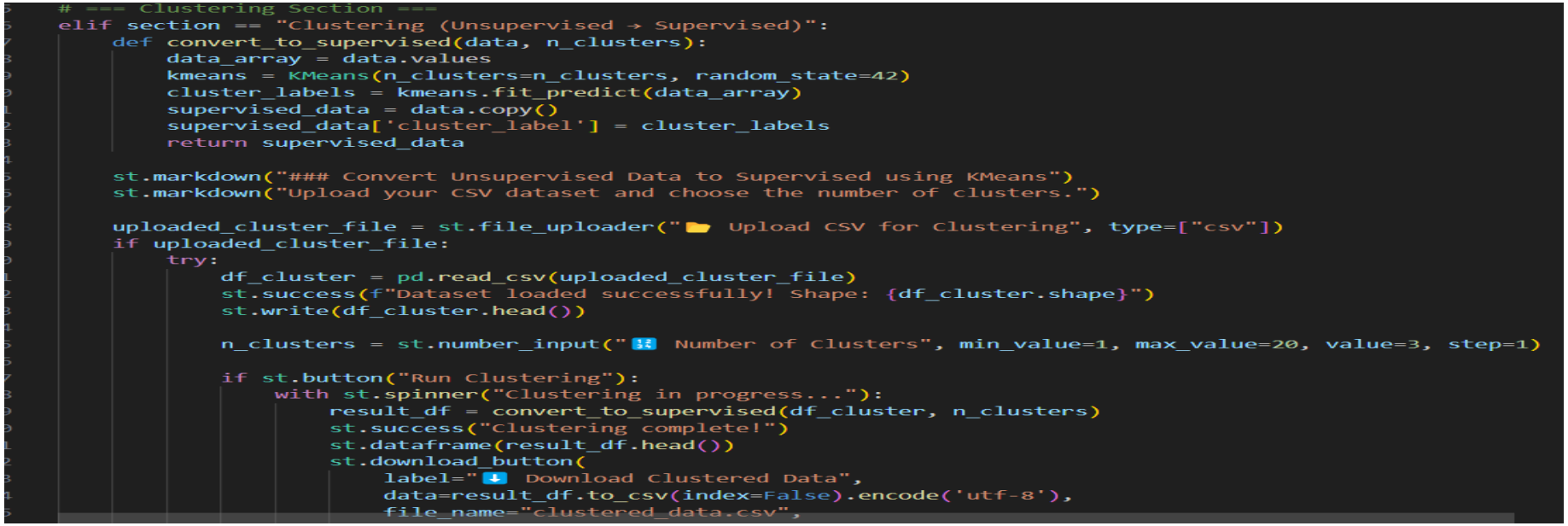
# Model Selection:

- Users upload a CSV with labeled data.
- Select the target column.
- Automatically sets up classification using PyCaret.
- Compares models and displays performance metrics using compare_models().

```
# === Model Selection Section ===
elif section == "Model Selection":
    if not os.path.exists("sourcefile.csv"):
        st.warning("Please upload and ingest a dataset first.")
    else:
        file = pd.read_csv("sourcefile.csv")
        task_type = st.selectbox("Select Task", ['Regression', 'Classification'])
        target = st.selectbox("Select Target Column", file.columns)
        if st.button("Run Modeling"):
            if task_type == "Regression":
                setup_df = setup_reg(file, target=target)
                st.subheader("Setup Summary")
                st.dataframe(pull_reg())
                best_model = compare_reg()
                st.subheader("Model Comparison")
                st.dataframe(pull_reg())
                evaluate_reg(best_model)
            else:
                setup_df = setup_clf(file, target=target)
                st.subheader("Setup Summary")
                st.dataframe(pull_clf())
                best_model = compare_clfs()
                st.subheader("Model Comparison")
                st.dataframe(pull_clf())
                evaluate_clfs(best_model)
```

# Clustering:

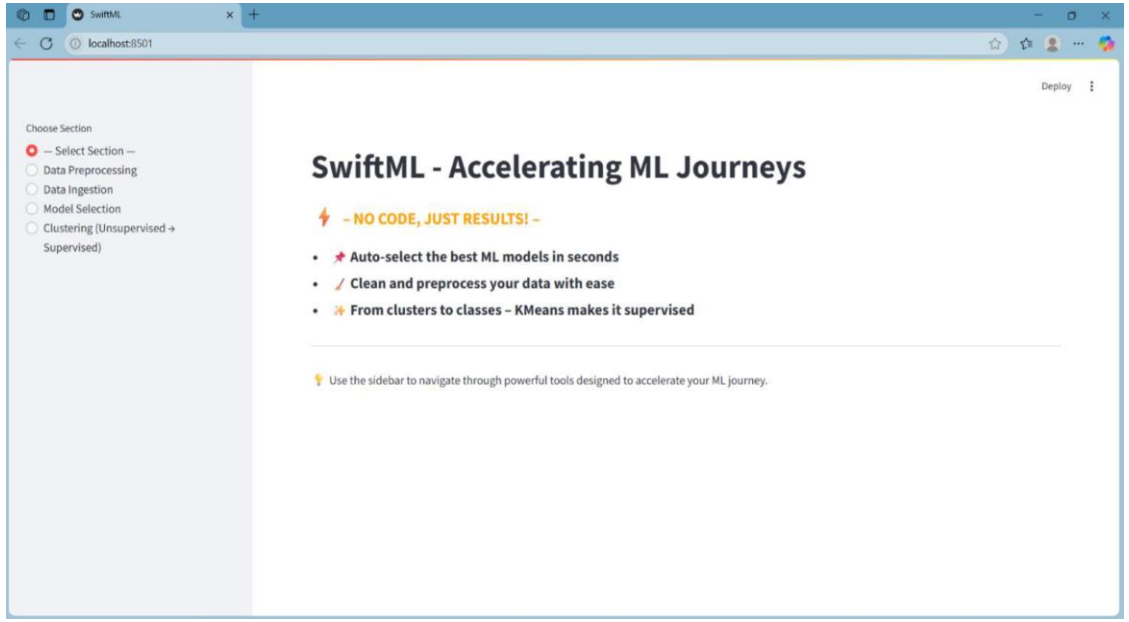
- Users upload a dataset.
- Choose clustering algorithm: KMeans or DBSCAN.
- Select columns to include.
- For KMeans: allows specifying number of clusters.
- Performs clustering and adds cluster labels to the data.
- Displays the data with the new cluster column.



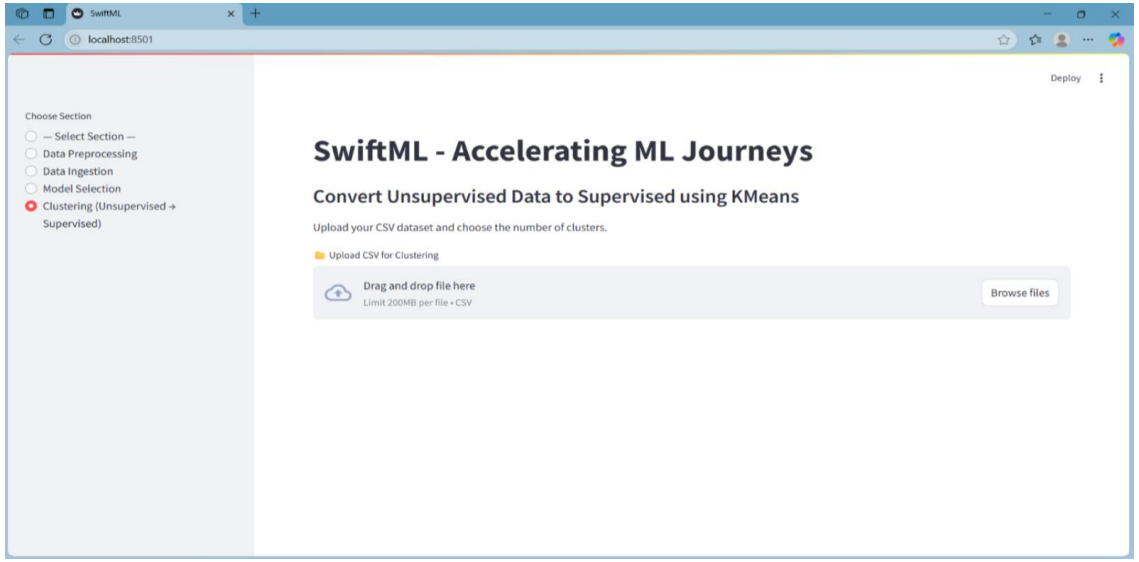
```
# === Clustering Section ===
elif section == "Clustering (Unsupervised → Supervised)":
    def convert_to_supervised(data, n_clusters):
        data_array = data.values
        kmeans = KMeans(n_clusters=n_clusters, random_state=42)
        cluster_labels = kmeans.fit_predict(data_array)
        supervised_data = data.copy()
        supervised_data['cluster_label'] = cluster_labels
        return supervised_data

    st.markdown("### Convert Unsupervised Data to Supervised using KMeans")
    st.markdown("Upload your CSV dataset and choose the number of clusters.")

    uploaded_cluster_file = st.file_uploader("📂 Upload CSV for Clustering", type=["csv"])
    if uploaded_cluster_file:
        try:
            df_cluster = pd.read_csv(uploaded_cluster_file)
            st.success(f"Dataset loaded successfully! Shape: {df_cluster.shape}")
            st.write(df_cluster.head())

            n_clusters = st.number_input("🔢 Number of Clusters", min_value=1, max_value=20, value=3, step=1)

            if st.button("Run Clustering"):
                with st.spinner("Clustering in progress..."):
                    result_df = convert_to_supervised(df_cluster, n_clusters)
                    st.success("Clustering complete!")
                    st.dataframe(result_df.head())
                    st.download_button(
                        label="⬇️ Download Clustered Data",
                        data=result_df.to_csv(index=False).encode('utf-8'),
                        file_name="clustered_data.csv",
```

# Results:

- Upload and clean their dataset
- Automatically select and evaluate the best ML models
- Apply clustering to convert unlabeled data into labelled data



- Clustering (Unsupervised → Supervised)
- This section turns unlabeled data into labeled data using KMeans clustering.
- You upload a dataset (without a target).
- Choose how many clusters you want (e.g., 3 groups).



# Conclusion and Future Enhancements:

SwiftML successfully builds the difference between complex machine learning processes and user access through the interface without code. This allows users to perform essential features such as data preprocessing, model selection and grouping with minimal technical effort. The structured workflows and automatic properties of the platform allow for quick use, making it ideal for students, analysts and professionals, who gain quick insights from data. The modular design ensures flexibility, while the spontaneous interface supports a smooth learning status for beginners in machine learning.

**Thoughts of future growth:**

**Model Paree's and Prediction:** Add features to store, load and distribute models with real-time prediction skills for end-to-end ML solutions.

**Advanced EDA and Preprying Tool**s: Visual Tools (Hemies, PCAS, Box Tomots) and Outliers, Feature Engineering and several options to handle more options for handling text/image data.

**User's individualization and clarity**: Users introduce the AI tools such as login, session story and better model transparency and user experience user experience.

# THANK YOU!